

Algorithmique Avancée – Travail Pratique (Version 4 Directions)

Le Problème du Labyrinthe (*Rat in a Maze*)

1. Contexte du problème

Une souris se trouve à la case départ (en haut à gauche) d'un labyrinthe représenté par une grille carrée en deux dimensions de taille $N \times N$. Elle doit trouver un chemin pour atteindre la case d'arrivée (située tout en bas à droite) afin d'y récupérer un morceau de fromage.

Le labyrinthe est modélisé par une liste de listes (matrice) d'entiers :

- Les cases contenant un **0** représentent les couloirs **libres** où la souris peut circuler.
- Les cases contenant un **1** représentent les **murs** infranchissables.

2. Règles de déplacement

La souris peut désormais explorer le labyrinthe dans les **quatre directions** cardinales :

1. Vers le **Bas** ($x + 1, y$)
2. Vers la **Droite** ($x, y + 1$)
3. Vers le **Haut** ($x - 1, y$)
4. Vers la **Gauche** ($x, y - 1$)

Il est strictement interdit de sortir des limites de la grille, de marcher sur un mur, ou de repasser par une case déjà occupée par le chemin en cours (sous peine de tourner en rond indéfiniment).

3. Objectif

Vous devez écrire une fonction Python `resoudre_labyrinthe(lab)` utilisant le paradigme du **Backtracking**.

Spécifications de la fonction :

- **Entrée** : Une matrice `lab` de taille $N \times N$ contenant des **0** et des **1**.
- **Sortie** : * Si un chemin existe, la fonction doit renvoyer une **nouvelle matrice** de taille $N \times N$ appelée `solution`. Dans cette matrice, les cases empruntées par le chemin final doivent valoir **1**, et toutes les autres doivent valoir **0**.
- Si aucun chemin n'est possible, la fonction doit renvoyer `None`.

⚠ Gestion des boucles et nettoyage : Votre algorithme doit veiller à ne pas revisiter une case déjà marquée à **1** dans la solution actuelle. De plus, si l'algorithme s'engage dans une impasse, il doit obligatoirement faire marche arrière et **effacer sa trace** (remettre la case à **0**) avant d'explorer une autre direction.

Tests mis à jour (4 Directions)

Voici la suite de tests pour valider votre implémentation. Copiez-collez ce code à la suite de votre fonction pour tester votre algorithme.

```
def extraire_case_arrivee(solution):
    """Renvoie la valeur de la case d'arrivée en bas à droite."""
    if not solution or isinstance(solution, str):
        return 0
    return solution[-1][-1]

def extraire_case_depart(solution):
    """Renvoie la valeur de la case de départ en haut à gauche."""
    if not solution or isinstance(solution, str):
        return 0
    return solution[0][0]

# =====
# TEST 1 : Cas de base (Tout ouvert)
# =====
lab_simple = [
    [0, 0],
    [0, 0]
]
res_1 = resoudre_labyrinthe(lab_simple)
print("Test 1...", "OK" if res_1 and extraire_case_arrivee(res_1) == 1 else "ÉCHEC")
assert res_1 and extraire_case_arrivee(res_1) == 1, "Échec Test 1 : Un labyrinthe simple et ouvert doit être résolu."

# =====
# TEST 2 : Pas de solution (Arrivée murée)
# =====
lab_bloque = [
    [0, 1],
    [0, 1]
]
res_2 = resoudre_labyrinthe(lab_bloque)
print("Test 2...", "OK" if res_2 is None else "ÉCHEC")
assert res_2 is None, "Échec Test 2 : Si l'arrivée est un mur, l'algorithme doit retourner None."

# =====
# TEST 3 : Départ bloqué immédiatement
# =====
lab_depart_bloque = [
    [1, 0],
    [0, 0]
]
res_3 = resoudre_labyrinthe(lab_depart_bloque)
print("Test 3...", "OK" if res_3 is None else "ÉCHEC")
```

```

assert res_3 is None, "Échec Test 3 : Si la case de départ est un mur, impossible
de commencer (renvoyer None)."
```

```

# =====
# TEST 4 : L'exemple classique du cours
# =====
lab_cours = [
    [0, 1, 0, 0],
    [0, 0, 1, 0],
    [1, 0, 0, 0],
    [1, 1, 1, 0]
]
res_4 = resoudre_labyrinthe(lab_cours)
print("Test 4...", "OK" if res_4 and res_4[1][1] == 1 and res_4[3][3] == 1 else
"ÉCHEC")
assert res_4 and res_4[1][1] == 1 and res_4[2][1] == 1 and res_4[3][3] == 1,
"Échec Test 4 : Le chemin trouvé n'est pas valide."

# =====
# TEST 5 : Le piège du cul-de-sac (Nettoyage requis)
# =====
lab_piege = [
    [0, 0, 0, 1],
    [1, 1, 0, 1],
    [0, 0, 0, 0],
    [0, 1, 1, 0]
]
res_5 = resoudre_labyrinthe(lab_piege)
print("Test 5...", "OK" if res_5 and res_5[0][3] == 0 else "ÉCHEC")
assert res_5 and res_5[0][1] == 1 and res_5[0][3] == 0, "Échec Test 5 : Le
backtracking n'a pas nettoyé les impasses."

# =====
# TEST 6 : Le chemin sinueux en "S" (Nécessite Haut et Gauche)
# =====
# Ce labyrinthe force la souris à monter et à aller à gauche pour s'en sortir
lab_sinueux = [
    [0, 0, 0, 0],
    [1, 1, 1, 0],
    [0, 0, 0, 0],
    [0, 1, 1, 1],
    [0, 0, 0, 0]
]
res_6 = resoudre_labyrinthe(lab_sinueux)
print("Test 6 (Chemin en S)...", "OK" if res_6 and extraire_case_arrivee(res_6) ==
1 else "ÉCHEC")
assert res_6 and extraire_case_arrivee(res_6) == 1, "Échec Test 6 : Impossible de
résoudre sans les mouvements Haut/Gauche."

# =====

```

```

# TEST 7 : Détection des boucles infinies
# =====
# Une grande pièce vide où l'algorithme pourrait tourner en rond si les cases
# visitées ne sont pas marquées.
lab_boucle = [
    [0, 0, 0],
    [0, 0, 0],
    [0, 0, 0]
]
try:
    res_7 = resoudre_labyrinthe(lab_boucle)
    print("Test 7 (Anti-boucle)...", "OK" if res_7 and
    extraire_case_arrivee(res_7) == 1 else "ÉCHEC")
except RecursionError:
    print("Test 7...", "ÉCHEC (Stack Overflow / Boucle infinie)")
    raise AssertionError("Échec Test 7 : Votre algorithme tourne en rond et a
    causé un Crash de Récursion !")

# =====
# TEST 8 : L'impasse initiale avec retour à zéro
# =====
lab_impasse_depart = [
    [0, 1, 1],
    [1, 1, 1],
    [1, 1, 0]
]
res_8 = resoudre_labyrinthe(lab_impasse_depart)
print("Test 8 (Impasse totale)...", "OK" if res_8 is None else "ÉCHEC")
assert res_8 is None, "Échec Test 8 : Devrait renvoyer None car le labyrinthe n'a
aucune issue."

# =====
# TEST 9 : Labyrinthe à Spirale (Spécifique 4 directions)
# =====
# Force l'algorithme à s'enrouler et à faire marche arrière si la cible finale
# n'était pas là.
lab_spirale = [
    [0, 0, 0, 0, 0],
    [1, 1, 1, 1, 0],
    [0, 0, 0, 1, 0],
    [0, 1, 0, 1, 0],
    [0, 1, 0, 0, 0]
]
res_9 = resoudre_labyrinthe(lab_spirale)
print("Test 9 (Spirale)...", "OK" if res_9 and extraire_case_arrivee(res_9) == 1
else "ÉCHEC")
assert res_9 and extraire_case_arrivee(res_9) == 1, "Échec Test 9 : Échec sur une
structure en spirale."

print("\nTous les tests du Labyrinthe (4 Directions) sont passés avec succès !")

```

